

Grammar non-terminals for postfix-expressions

Document #: P4176R1 [[Latest](#)] [[Status](#)]
Date: 2026-05-12
Project: Programming Language C++
Audience: Core Working Group
Reply-to: Vlad Serebrennikov
[<serebrennikov.vladislav@gmail.com>](mailto:serebrennikov.vladislav@gmail.com)

Contents

- [1 Abstract](#)
- [2 Revision history](#)
- [3 Approach](#)
- [4 Proposed wording](#)
 - [4.1 Postfix expressions](#) [\[expr.post\]](#)
 - [4.1.1 General](#) [\[expr.post.general\]](#)
 - [4.1.2 Subscripting](#) [\[expr.sub\]](#)
 - [4.1.3 Function call](#) [\[expr.call\]](#)
 - [4.1.4 Explicit type conversion \(functional notation\)](#) [\[expr.type.conv\]](#)
 - [4.1.5 Class member access](#) [\[expr.ref\]](#)
 - [4.1.6 Increment and decrement](#) [\[expr.post.incr\]](#)
 - [4.1.7 Dynamic cast](#) [\[expr.dynamic.cast\]](#)
 - [4.1.8 Type identification](#) [\[expr.typeid\]](#)
 - [4.1.9 Static cast](#) [\[expr.static.cast\]](#)
 - [4.1.10 Reinterpret cast](#) [\[expr.reinterpret.cast\]](#)
 - [4.1.11 Const cast](#) [\[expr.const.cast\]](#)
 - [4.2 Overload resolution](#) [\[over\]](#)
 - [4.2.1 Function call syntax](#) [\[over.match.call.general\]](#)
 - [4.2.2 Call to designated function](#) [\[over.call.func\]](#)

— 4.2.3 Call to object of class type	[over.call.object]
— 4.2.4 Function call	[over.call]
— 4.2.5 Subscripting	[over.sub]
— 4.2.6 Class member access	[over.ref]
— 4.3 The rest of the core language wording	
— 4.3.1 One-definition rule	[basic.def.odr]
— 4.3.2 Unqualified name lookup	[basic.lookup.unqual]
— 4.3.3 Argument-dependent name lookup	[basic.lookup.argdep]
— 4.3.4 Qualified name lookup	[basic.lookup.qual.general]
— 4.3.5 Dynamic storage duration	[basic.stc.dynamic]
— 4.3.6 Sequential execution	[intro.execution]
— 4.3.7 Value category	[basic.lval]
— 4.3.8 Parentheses	[expr.prim.paren]
— 4.3.9 Names	[expr.prim.id.general]
— 4.3.10 Unqualified names	[expr.prim.id.unqual]
— 4.3.11 Destruction	[expr.prim.id.dtor]
— 4.3.12 Expression splicing	[expr.prim.splice]
— 4.3.13 Unary operators	[expr.unary.op]
— 4.3.14 Comma operator	[expr.comma]
— 4.3.15 Core constant expressions	[expr.const.core]
— 4.3.16 Default arguments	[dcl.fct.default]
— 4.3.17 Defaulted comparison operator functions	[class.compare.default]
— 4.3.18 Names of template specialization	[temp.names]
— 4.3.19 Dependent names	[temp.dep.general]
— 4.3.20 Type-dependent expressions	[temp.dep.expr]
— 4.3.21 Value-dependent expression	[temp.dep.constexpr]
— 4.3.22 Deducing template arguments from a type	[temp.deduct.type]
— 4.3.23 Exception specifications	[except.spec]
— 4.4 Library wording	
— 4.4.1 Algorithm function objects	[alg.func.obj]
— 4.4.2 Creation	[support.srcloc.cons]
— 4.4.3 Function objects	[function.objects.general]

1 Abstract

7.6.1 [expr.post] lists all the grammar at the top, while the rest of the subclause resorts to either quoting the grammar or describing it with words. This paper aims to improve the situation, introducing new non-terminals and putting their definitions in the respective subclauses.

2 Revision history

R1:

- Adjust proposed wording to address feedback from Jens Maurer.
- Address feedback from CWG telecon on 2026-04-28:
 - Remove (italicized) definition of “function call”.
 - Replace usages of “subscript expression” and “class member access expression” with the respective grammar terms.
 - Remove introductory sentences in [expr.sub] and [expr.call].
 - Integrate [expr.post.incr]/2 into [expr.post.incr]/1.
 - Add new wording to [expr.prim.paren] to explain that “(possibly parenthesized)” covers more than one level of parentheses.

R0:

- Initial revision.

3 Approach

- Move grammar definitions of various forms of *postfix-expression* from 7.6.1.1 [expr.-post.general] to their respective subclauses, defining new non-terminals.
- Define *postfix-expressions* in terms of new non-terminals.
- Make adjustments to the surrounding wording to improve integration between grammar and description of the semantics.
- Replace usages of regular font terms with their corresponding grammar productions. For instance, usages of “class member access expression” are replaced with *class-member-access-expression*.

No changes to behavior of programs are intended.

4 Proposed wording

Hide removals: ☐
Strikethrough: ☒

§ 4.1 Postfix expressions

[[expr.post](#)]

§ 4.1.1 General

[[expr.post.general](#)]

Change 7.6.1.1 [[expr.post.general](#)] paragraph 1 as follows:

- 1 Postfix expressions group left-to-right.

```
postfix-expression:  
  primary-expression  
postfix-expression [expression-listopt]→  
  subscript-expression  
postfix-expression [expression-listopt]→  
  function-call-expression  
simple-type-specifier [expression-listopt]→  
typename-specifier [expression-listopt]→  
simple-type-specifier braced-init-list  
typename-specifier braced-init-list  
  type-conversion-expression  
postfix-expression templateopt id-expression  
postfix-expression splice-expression  
postfix-expression → templateopt id-expression  
postfix-expression → splice-expression  
  class-member-access-expression  
postfix-expression ++  
  post-increment-expression  
postfix-expression --  
  post-decrement-expression  
dynamic_cast < type-id > [expression]→  
  dynamic-cast-expression  
static_cast < type-id > [expression]→  
  static-cast-expression  
reinterpret_cast < type-id > [expression]→  
  reinterpret-cast-expression  
const_cast < type-id > [expression]→  
  const-cast-expression  
typeid [expression]→  
typeid [type-id]→  
  typeid-expression  
  
expression-list:  
  initializer-list
```

Remove 7.6.1.1 [[expr.post.general](#)] paragraph 2:

- 2 [*Note: The ~~→~~ token following the *type-id* in a *dynamic_cast*, *static_cast*, *reinterpret_cast*, or *const_cast* can be the product of replacing a ~~→~~ token by two consecutive ~~→~~ tokens (13.3 [[temp.names](#)]).* — end note]

[Drafting note: A similar note is added to each kind of cast.]

§ 4.1.2 Subscripting

[expr.sub]

Change 7.6.1.2 [expr.sub] paragraph 1 as follows:

subscript-expression:
postfix-expression [*expression-list*_{opt}]

- 1 ~~A subscript expression is a postfix expression followed by square brackets containing a possibly empty, comma-separated list of initializer-clauses that~~ The expression-list, if present, constitutes the arguments to the subscript operator. The *postfix-expression* and the initialization of the object parameter (9.3.4.6 [dcl.fct]) of any applicable subscript operator function (12.4.5 [over.sub]) is sequenced before each expression in the *expression-list* and also before any default argument (9.3.4.7 [dcl.fct.default]). The initialization of a non-object parameter of a subscript operator function *S*, including every associated value computation and side effect, is indeterminately sequenced with respect to that of any other non-object parameter of *S*.

§ 4.1.3 Function call

[expr.call]

Change 7.6.1.3 [expr.call] paragraph 1 as follows:

function-call-expression:
postfix-expression (*expression-list*_{opt})

- 1 ~~A function call is a postfix expression followed by parentheses containing a possibly empty, comma-separated list of initializer-clauses which~~ The expression-list, if present, constitutes the arguments to ~~the a~~ function designated by the postfix-expression.

[Note: If the ~~postfix-expression~~ *postfix-expression* is a function name, the appropriate function and the validity of the call are determined according to the rules in 12.2 [over.match]. — end note]

The ~~postfix-expression~~ *postfix-expression* *E* shall have function type or function pointer type. For a call to a non-member function or to a static member function, ~~the postfix-expression~~ *E* shall be either an lvalue that refers to a function (in which case the function-to-pointer standard conversion (7.3.4 [conv.func]) is suppressed on ~~the postfix-expression~~ *E*), or a prvalue of function pointer type.

Change 7.6.1.3 [expr.call] paragraph 2 as follows:

- 2 If the selected function is non-virtual, or if ~~the id-expression in the class member access-expression~~ *E* is a class-member-access-expression whose id-expression is a *qualified-id*, that function is called. Otherwise, its final overrider (11.7.3 [class.virtual]) in the dynamic type of the object expression is called; such a call is referred to as a *virtual function call*.

§ 4.1.4 Explicit type conversion (functional notation)[[expr.type.conv](#)]

Change 7.6.1.4 [[expr.type.conv](#)] paragraph 1 as follows:

type-conversion-expression:
simple-type-specifier (*expression-list*_{opt})
typename-specifier (*expression-list*_{opt})
simple-type-specifier *braced-init-list*
typename-specifier *braced-init-list*

1 ~~A *simple-type-specifier* or *typename-specifier* followed by a parenthe-~~
~~sized optional *expression-list* or by a *braced-init-list* (the initializer)~~ A
type-conversion-expression constructs a value of the ~~specified~~ type
specified by the *simple-type-specifier* or *typename-specifier*. ~~given~~ The
braced-init-list or parentheses containing optional *expression-list* is
the initializer. If the type is a placeholder for a deduced class type,
it is replaced by the return type of the function selected by overload
resolution for class template deduction for the remainder of this sub-
clause. Otherwise, if the type contains a placeholder type, it is re-
placed by the type determined by placeholder type deduction
(9.2.9.7.2 [[decl.type.auto.deduct](#)]). Let T denote the resulting type.
Then:

(1.1) — If the initializer is a parenthesized ~~single expression~~ *expression-*
list with a single assignment-expression, the type conversion ex-
pression is equivalent to the corresponding cast expression
(7.6.3 [[expr.cast](#)]).

(1.2) — Otherwise, if T is *cv void*, the initializer shall be ~~()~~ or ~~{}~~ (af-
ter pack expansion, if any), and the expression is a prvalue of
type *void* that performs no initialization.

(1.3) — Otherwise, if T is a reference type, the expression has the same
effect as direct-initializing an invented variable *t* of type T
from the initializer and then using *t* as the result of the ex-
pression; the result is an lvalue if T is an lvalue reference type
or an rvalue reference to function type and an xvalue
otherwise.

(1.4) — Otherwise, the expression is a prvalue of type T whose result
object is direct-initialized (9.5 [[decl.init](#)]) with the initializer.

If the initializer is a parenthesized optional *expression-list*, T shall not
be an array type.

[Example: [. . .] — end example]

[Drafting note: Do we want to use “(possibly parenthesized)” here, given “the initializer shall be () or {}” in 1.2?]

§ 4.1.5 Class member access

[[expr.ref](#)]

Change 7.6.1.5 [expr.ref] paragraph 1 as follows:

```
class-member-access-expression:  
    postfix-expression . templateopt id-expression  
    postfix-expression . splice-expression  
    postfix-expression -> templateopt id-expression  
    postfix-expression -> splice-expression
```

- 1 ~~A postfix expression followed by a dot `.` or an arrow `->`, optionally followed by the keyword `template`, and then followed by an *id-expression* or a *splice-expression*, is a postfix expression.~~

[Note: If the keyword `template` is used and followed by an *id-expression*, the unqualified name is considered to refer to a template (13.3 [temp.names]). If a *simple-template-id* results and is followed by a `::`, the *id-expression* is a *qualified-id*. — end note]

[Drafting note: This note is under consideration by CWG3180.]

Change footnote 45 referenced from 7.6.1.5 [expr.ref] paragraph 3:

If the ~~class member access expression~~ class-member-access-expression is evaluated, the subexpression evaluation happens even if the result is unnecessary to determine the value of the entire postfix expression, for example if the *id-expression* denotes a static member.

§ 4.1.6 Increment and decrement

[expr.post.incr]

Change 7.6.1.6 [expr.post.incr] paragraph 1 as follows:

```
post-increment-expression  
    postfix-expression ++  
  
post-decrement-expression  
    postfix-expression --
```

- 1 The value of ~~a postfix ++ expression~~ a *post-increment-expression* or *post-decrement-expression* *E* is the value obtained by applying the lvalue-to-rvalue conversion (7.3.2 [conv.lval]) to its operand, which is the *postfix-expression*.

[Note: The value obtained is a copy of the original value. — end note]

The operand shall be a modifiable lvalue. The type of the operand shall be an arithmetic type other than *cv bool*, or a pointer to a complete object type. An operand with volatile-qualified type is deprecated; see D.4 [depr.volatile.type]. The value of the operand object is modified (3.1 [defs.access]) as if it were the operand of

- (1.1) — the prefix `++` operator (7.6.2.3 [expr.pre.incr]) if *E* is a *post-increment-expression*, or
- (1.2) — the prefix `--` operator if *E* is a *post-decrement-expression*.

The value computation of ~~the ++expression~~ *E* is sequenced before the modification of the operand object. With respect to an indeterminately-sequenced function call, the operation of ~~postfix ++~~ *E* is a single evaluation.

[*Note:* Therefore, a function call cannot intervene between the lvalue-to-rvalue conversion and the side effect associated with any single ~~postfix ++ operator~~ *post-increment-expression* or *post-decrement-expression*. — end note]

The result is a prvalue. The type of the result is the cv-unqualified version of the type of the operand.

Remove 7.6.1.6 [expr.post.incr] paragraph 2:

~~2 The operand of postfix -- is decremented analogously to the postfix ++ operator.~~

~~[*Note:* For prefix increment and decrement, see 7.6.2.3 [expr.pre.incr]. — end note]~~

§ 4.1.7 Dynamic cast

[expr.dynamic.cast]

Change 7.6.1.7 [expr.dynamic.cast] paragraph 1 as follows:

dynamic-cast-expression:

`dynamic_cast < type-id > ( expression )`

1 The result of ~~the expression dynamic_cast<T>(v)~~ a *dynamic-cast-expression* is the result of converting the ~~expression~~ *expression* *v* to the type *T* designated by the type-id. *T* shall be a pointer or reference to a complete class type, or “pointer to *cv void*”. The `dynamic_cast` operator shall not cast away constness (7.6.1.11 [expr.const.cast]).

[*Note:* The > token following the *type-id* can be the product of replacing a >> token by two consecutive > tokens (13.3 [temp.names]). — end note]

§ 4.1.8 Type identification

[expr.typeid]

Change 7.6.1.8 [expr.typeid] paragraph 1 as follows:

typeid-expression

`typeid ( expression )`

`typeid ( type-id )`

1 The result of a ~~typeid-expression~~ *typeid-expression* is an lvalue of static type `const std::type_info` (17.7.3 [type.info]) and dynamic type `const std::type_info` or `const name` where *name* is an implementation-defined class publicly derived from `std::type_info` which preserves the behavior described in 17.7.3 [type.info].⁴⁸ The lifetime of the object referred to by the lvalue extends to the end of

the program. Whether or not the destructor is called for the `std::type_info` object at the end of the program is unspecified.

§ 4.1.9 Static cast

[[expr.static.cast](#)]

Change 7.6.1.9 [[expr.static.cast](#)] paragraph 1 as follows:

```
static-cast-expression  
static_cast < type-id > ( expression )
```

- 1 The result of ~~the expression *static_cast*<T>(v)~~ a *static-cast-expression* is the result of converting the ~~expression~~ *expression* v to the type T designated by the *type-id*. If T is an lvalue reference type or an rvalue reference to function type, the result is an lvalue; if T is an rvalue reference to object type, the result is an xvalue; otherwise, the result is a prvalue.

[*Note:* The > token following the *type-id* can be the product of replacing a >> token by two consecutive > tokens (13.3 [[temp.names](#)]). — end note]

§ 4.1.10 Reinterpret cast

[[expr.reinterpret.cast](#)]

Change 7.6.1.10 [[expr.reinterpret.cast](#)] paragraph 1 as follows:

```
reinterpret-cast-expression  
reinterpret_cast < type-id > ( expression )
```

- 1 The result of ~~the expression *reinterpret_cast*<T>(v)~~ a *reinterpret-cast-expression* is the result of converting the ~~expression~~ *expression* v to the type T designated by the *type-id*. If T is an lvalue reference type or an rvalue reference to function type, the result is an lvalue; if T is an rvalue reference to object type, the result is an xvalue; otherwise, the result is a prvalue and the lvalue-to-rvalue, array-to-pointer, and function-to-pointer standard conversions are performed on the expression v. Conversions that can be performed explicitly using `reinterpret_cast` are listed below. No other conversion can be performed explicitly using `reinterpret_cast`.

[*Note:* The > token following the *type-id* can be the product of replacing a >> token by two consecutive > tokens (13.3 [[temp.names](#)]). — end note]

§ 4.1.11 Const cast

[[expr.const.cast](#)]

Change 7.6.1.11 [[expr.const.cast](#)] paragraph 1 as follows:

```
const-cast-expression  
const_cast < type-id > ( expression )
```

- 1 ~~The result of the *const_cast*<T>(v) is of type T.~~ The result of a *const-cast-expression* is the result of converting the *expression* v to the type T designated by the *type-id*. If T is an lvalue reference to

object type, the result is an lvalue; if T is an rvalue reference to object type, the result is an xvalue; otherwise, the result is a prvalue and the lvalue-to-rvalue, array-to-pointer, and function-to-pointer standard conversions are performed on the expression v. The temporary materialization conversion is not performed on v, other than as specified below. Conversions that can be performed explicitly using `const_cast` are listed below. No other conversion shall be performed explicitly using `const_cast`.

[Note: The > token following the *type-id* can be the product of replacing a >> token by two consecutive > tokens (13.3 [temp.names]). — end note]

§ 4.2 Overload resolution [over]

§ 4.2.1 Function call syntax [over.match.call.general]

Change 12.2.2.2.1 [over.match.call.general] paragraph 1 as follows:

- 1 ~~In a function call (7.6.1.3 [expr.call])~~
~~*postfix-expression* (~~*expression-list*~~_{opt})~~
 if If the *postfix-expression* of a function-call-expression names at least one function or function template, overload resolution is applied as specified in 12.2.2.2.2 [over.call.func]. If the *postfix-expression* denotes an object of class type, overload resolution is applied as specified in 12.2.2.2.3 [over.call.object].

§ 4.2.2 Call to designated function [over.call.func]

Apply the following changes to the entire subclause 12.2.2.2.2 [over.call.func]:

- 1 ~~Of interest in 12.2.2.2.2 [over.call.func] are only those function calls in which the *postfix-expression* ultimately contains an *id-expression* or *splice-expression* that designates one or more functions. Such a *postfix-expression*, perhaps nested arbitrarily deep in parentheses, has one of the following forms:~~
~~*postfix-expression*:~~
~~*postfix-expression* ~~*·*~~ *id-expression*~~
~~*postfix-expression* ~~*·*~~ *splice-expression*~~
~~*postfix-expression* ~~*→*~~ *id-expression*~~
~~*postfix-expression* ~~*→*~~ *splice-expression*~~
~~*id-expression*~~
~~*splice-expression*~~
~~These represent two syntactic subcategories of function calls: qualified function calls and unqualified function calls.~~
- 2 ~~In qualified function calls, the function is designated by an A qualified function call is a *function-call-expression* whose (possibly parenthesized) *postfix-expression* is a *class-member-access-expression* whose *id-expression* or *splice-expression* *E* preceded by an ~~*→*~~ or ~~*·*~~ operator~~

designates a function. Since the construct $A \rightarrow B$ is generally equivalent to $(*A).B$, the rest of Clause 12 [over] assumes, without loss of generality, that all member function calls have been normalized to the form that uses an object and the `.` operator. Furthermore, Clause 12 [over] assumes that the *postfix-expression* that is the left operand of the `.` operator has type “*cv T*” where *T* denotes a class.⁹⁴ The set of candidate functions either is the set found by name lookup (6.5.2 [class.member.lookup]) if *E* is an *id-expression* or is the set determined as specified in 7.5.9 [expr.prim.splice] if *E* is a *splice-expression*. The argument list ~~is the expression-list in the call~~ consists of the arguments of the function-call-expression augmented by the addition of the left operand of the `.` operator in the normalized member function call as the implied object argument (12.2.2 [over.-match.funcs]).

- 3 ~~In unqualified function calls, the function is designated by~~ An unqualified function call is a function-call-expression whose (possibly parenthesized) postfix-expression is an id-expression or a splice-expression *E* which designates a function. The set of candidate functions either is the set found by name lookup (6.5 [basic.lookup]) if *E* is an *id-expression* or is the set determined as specified in 7.5.9 [expr.prim.splice] if *E* is a *splice-expression*. The set of candidate functions consists either entirely of non-member functions or entirely of member functions of some class *T*. In the former case or if *E* is either a *splice-expression* or the address of an overload set, the argument list ~~is the same as the expression-list in the call~~ consists of the arguments of the function-call-expression. Otherwise, the argument list ~~is the expression-list in the call~~ consists of the arguments of the function-call-expression augmented by the addition of an implied object argument as in a qualified function call. [`. . .`]

[Example: [`. . .`] — end example]

[Drafting note: Does the definition of unqualified function call cover `sizeof((**func)())`;, which is accepted by all implementations (Compiler Explorer)?]

§ 4.2.3 Call to object of class type

[over.call.object]

Change 12.2.2.2.3 [over.call.object] paragraph 1 as follows:

- 1 If the *postfix-expression* *E* ~~in the function call syntax of a function-call-expression~~ evaluates to a class object of type “*cv T*”, then the set of candidate functions includes at least the function call operators of *T*. The function call operators of *T* are the results of a search for the name `operator()` in the scope of *T*.

§ 4.2.4 Function call

[over.call]

Change 12.4.4 [over.call] paragraph 1 as follows:

- 1 A *function call operator function* is a function named `operator()` that is a member function with an arbitrary number of parameters. It may have default arguments. For ~~an expression of the form~~ a *function-call-expression*

postfix-expression (*expression-list*_{opt})

~~where~~ whose the *postfix-expression* is of class type, the operator function is selected by overload resolution (12.2.2.2.3 [over.call.object]). If a surrogate call function is selected, let *e* be the result of invoking the corresponding conversion operator function on the *postfix-expression*;

the expression is interpreted as

e (*expression-list*_{opt})

Otherwise, the expression is interpreted as

postfix-expression . `operator` () (*expression-list*_{opt})

[Drafting note: Should we keep quoting the grammar of *function-call-expression* to make it clear where *expression-list* comes from? The same applies to [over.sub] changes below.]

§ 4.2.5 Subscripting

[over.sub]

Change 12.4.5 [over.sub] paragraph 1 as follows:

- 1 A *subscripting operator function* is a member function named `operator[]` with an arbitrary number of parameters. It may have default arguments. For ~~an expression of the form~~ a *subscript-expression*

postfix-expression [*expression-list*_{opt}]

the operator function is selected by overload resolution (12.2.2.3 [over.match.oper]). If a member function is selected, the expression is interpreted as

postfix-expression . `operator` [] (*expression-list*_{opt})

§ 4.2.6 Class member access

[over.ref]

Change 12.4.6 [over.ref] paragraph 1 as follows:

- 1 A *class member access operator function* is a function named `operator->` that is a non-static member function taking no non-object parameters. For ~~an expression~~ a *class-member-access-expression* of the form

postfix-expression -> `template`_{opt} *id-expression*

the operator function is selected by overload resolution (12.2.2.3 [over.match.oper]), and the expression is interpreted as

(*postfix-expression* . `operator` -> ()) -> `template`_{opt} *id-expression*

Analogously, for ~~an expression~~ a class-member-access-expression of the form

postfix-expression $\rightarrow$ *splice-expression*

the operator function is selected by overload resolution, and the expression is interpreted as

(*postfix-expression* . *operator* $\rightarrow$ ()) $\rightarrow$ *splice-expression*

§ 4.3 The rest of the core language wording

§ 4.3.1 One-definition rule

[basic.def.odr]

Change 6.3 [basic.def.odr] paragraph 3 as follows:

3 An expression or conversion is *potentially evaluated* unless it is an unevaluated operand (7.2.3 [expr.context]), a subexpression thereof, or a conversion in an initialization or conversion sequence in such a context. The set of *potential results* of an expression E is defined as follows:

- [. . .]
- (3.3) — If E is a ~~class member access expression (7.6.1.5)~~ class-member-access-expression of the form E_1 . *template*_{opt} E_2 , where E_2 designates a non-static data member or a direct base class relationship, the set contains the potential results of E_1 .
- (3.4) — If E is a ~~class member access expression~~ class-member-access-expression naming a static data member, the set contains the *id-expression* designating the data member.
- [. . .]

Change 6.3 [basic.def.odr] paragraph 5 as follows:

5 A variable is named by an expression if the expression is an *id-expression* or *splice-expression* (7.5.9 [expr.prim.splice]) that designates it. A variable x that is named by a potentially evaluated expression N that appears at a point P is *odr-used* by N unless

- (5.1) — x is a reference that is usable in constant expressions at P ([expr.const.init]), or
- (5.2) — N is an element of the set of potential results of an expression E , where
 - (5.2.1) — E is a discarded-value expression (7.2.3 [expr.context]) to which the lvalue-to-rvalue conversion is not applied, or
 - (5.2.2) — x is a non-volatile object that is usable in constant expressions at P and has no mutable subobjects, and
 - (5.2.2.1) — E is a ~~class member access expression (7.6.1.5)~~ [expr.pr.ref] class-member-access-expression naming a

- (5.2.2.2) non-static data member of reference type and whose object expression has non-volatile-qualified type, or
- the lvalue-to-rvalue conversion (7.3.2 [conv.lval]) is applied to *E* and *E* has non-volatile-qualified non-class type.
- [Example: [. . .] — end example]

Change 6.3 [basic.def.odr] paragraph 7 as follows:

- 7 ***this** is odr-used if **this** appears as a potentially evaluated expression (including as the result of any implicit transformation to a ~~class member access expression~~ class-member-access-expression (7.5.5.1 [expr.prim.id.general])).

§ 4.3.2 Unqualified name lookup [basic.lookup.unqual]

Change 6.5.3 [basic.lookup.unqual] paragraph 4 as follows:

- 4 An *unqualified name* is a name that does not immediately follow a *nested-name-specifier* or the `.` or `->` in a ~~class member access expression~~ (7.6.1.5) class-member-access-expression, possibly after a **template** keyword or `~`. Unless otherwise specified, such a name undergoes unqualified name lookup from the point where it appears.

§ 4.3.3 Argument-dependent name lookup [basic.lookup.argdep]

Change 6.5.4 [basic.lookup.argdep] paragraph 1 as follows:

- 1 When the *postfix-expression* in a ~~function call~~ (7.6.1.3 [expr.call]) function-call-expression is an *unqualified-id*, and unqualified lookup (6.5.3 [basic.lookup.unqual]) for the name in the *unqualified-id* does not find any
- (1.1) — declaration of a class member, or
 - (1.2) — function declaration inhabiting a block scope, or
 - (1.3) — declaration not of a function or function template
- then lookup for the name also includes the result of *argument-dependent lookup* in a set of associated namespaces that depends on the types of the arguments (and for type template template arguments, the namespace of the template argument), as specified below.
- [Example: [. . .] — end example]

Change 6.5.4 [basic.lookup.argdep] paragraph 2 as follows:

[*Note:* For purposes of determining (during parsing) whether an expression is a *postfix-expression* ~~for a function call~~ *of a function-call-expression*, the usual name lookup rules apply. In some cases a name followed by `<` is treated as a *template-name* even though name lookup did not find a *template-name* (see 13.3 [temp.names]). For example,

```
int h;
void g();
namespace N {
    struct A {};
    template <class T> int f(T);
    template <class T> int g(T);
    template <class T> int h(T);
}

int x = f<N::A>(N::A());           // OK, lookup of f finds nothing, f
                                   treated as template name
int y = g<N::A>(N::A());           // OK, lookup of g finds a function,
                                   g treated as template name
int z = h<N::A>(N::A());           // error: h< does not begin a tem-
                                   plate-id
```

The rules have no effect on the syntactic interpretation of an expression. For example,

```
typedef int f;
namespace N {
    struct A {
        friend void f(A &);
        operator int();
        void g(A a) {
            int i = f(a);           // f is the typedef, not the friend
                                   function: equivalent to int(a)
        }
    };
}
```

Because the expression is not a *function-call* *function-call-expression*, argument-dependent name lookup does not apply and the friend function `f` is not found. — *end note*

§ 4.3.4 Qualified name lookup

[basic.lookup.qual.general]

Change 6.5.5.1 [basic.lookup.qual.general] paragraph 2 as follows:

- 2 A *member-qualified name* is the (unique) component name (7.5.5.2 [expr.prim.id.unqual]), if any, of
 - (2.1) — an *unqualified-id* or
 - (2.2) — a *nested-name-specifier* of the form *type-name* `::` or *namespace-name* `::`

in the *id-expression* of a ~~class member access expression~~ (7.6.1.5 [ex-
pr.ref]) *class-member-access-expression*. [. . .]

[Note: [. . .] — end note]

[Example: [. . .] — end example]

§ 4.3.5 Dynamic storage duration

[basic.stc.dynamic]

Change 6.8.6.5.1 [basic.stc.dynamic.general] paragraph 2 as follows:

2 [. . .]

[Note: the implicit declarations do not introduce the names `std`, `std::size_t`, `std::align_val_t`, or any other names that the library uses to declare these names. Thus, a *new-expression*, *delete-expression*, or ~~function-call~~ *function-call-expression* that refers to one of these functions without importing or including the header `<new>` (17.6.2 [new.syn]) or importing a C++ library module (16.4.2.4 [std.modules]) is well-formed. However, referring to `std` or `std::size_t` or `std::align_val_t` is ill-formed unless a standard library declaration (17.2.1 [cstddef.syn], 17.6.2 [new.syn], 16.4.2.4 [std.modules]) of that name precedes (6.5.1 [basic.lookup.general]) the use of that name. — end note]

[. . .]

§ 4.3.6 Sequential execution

[intro.execution]

Change 6.10.1 [intro.execution] paragraph 11 as follows:

- 11 When invoking a function f (whether or not the function is inline), every argument expression and the ~~postfix-expression~~ *postfix-expression* designating f are sequenced before every precondition assertion of f (9.4.1 [dcl.contract.func]), which in turn are sequenced before every expression or statement in the body of f , which in turn are sequenced before every postcondition assertion of f .

§ 4.3.7 Value category

[basic.lval]

Change 7.2.1 [basic.lval] paragraph 4 as follows:

[Note: An expression is an xvalue if it is:

— [. . .]

- (4.5) — a ~~class member access expression~~ *class-member-access-expression* designating a non-static data member of non-reference type in which the object expression is an xvalue (7.6.1.5 [expr.ref]), or

— [. . .]

[. . .]

[Example: [. . .] — end example]

— end note]

Add a new paragraph at the end of 7.5.4 [expr.prim.paren]:

- 1 A parenthesized expression (*E*) is a primary expression whose type, result, and value category are identical to those of *E*. The parenthesized expression can be used in exactly the same contexts as those where *E* can be used, and with the same meaning, except as otherwise indicated.
- n A possibly parenthesized expression may be nested arbitrarily deep in parentheses.

[Drafting note: Do we need a cross-reference to [implimits]/1.4?]

Change 7.5.5.1 [expr.prim.id.general] paragraph 2 as follows:

- 2 If an *id-expression* *E* denotes a non-static non-type member of some class *C* at a point where the current class (7.5.3 [expr.prim.this]) is *X* and
 - (2.1) — *E* is potentially evaluated or *C* is *X* or a base class of *X*, and
 - (2.2) — *E* is not the *id-expression* of a ~~class member access expression~~ (7.6.1.5 [expr.ref]) class-member-access-expression, and
 - (2.3) — *E* is not the *id-expression* of a *reflect-expression* (7.6.2.10 [expr.reflect]), and
 - (2.4) — if *E* is a *qualified-id*, *E* is not the un-parenthesized operand of the unary *&* operator (7.6.2.2 [expr.unary.op]),

the *id-expression* is transformed into a ~~class member access expression~~ class-member-access-expression using (**this*) as the object expression. If this transformation occurs in the predicate of a precondition assertion of a constructor of *X* or a postcondition assertion of a destructor of *X*, the expression is ill-formed.

[Note: If *C* is not *X* or a base class of *X*, the ~~class member access expression~~ class-member-access-expression is ill-formed. Also, if the *id-expression* occurs within a static or explicit object member function, the class member access is ill-formed. — end note]

This transformation does not apply in the template definition context (13.8.3.2 [temp.dep.type]).

[Example: [. . .] — end example]

Change 7.5.5.1 [expr.prim.id.general] paragraph 3 as follows:

- 3 If an *id-expression* *E* denotes a variant member *M* of an anonymous union (11.5.2 [class.union.anon]) *U*:

- (3.1) — If U is a non-static data member, E refers to M as a member of the lookup context of the terminal name of E (after any implicit transformation to a ~~class member access expression~~ class-member-access-expression).
 [Example: $o.x$ is interpreted as $o.u.x$, where u names the anonymous union member. — end example]
- [. . .]

§ 4.3.10 Unqualified names

[[expr.prim.id.unqual](#)]

Change 7.5.5.2 [[expr.prim.id.unqual](#)] paragraph 4 as follows:

- 4 if
- (4.1) — the *unqualified-id* appears in a *lambda-expression* at program point P ,
- (4.2) — the entity is a local entity (6.1 [[basic.pre](#)]) or a variable declared by an *init-capture* (7.5.6.3 [[expr.prim.lambda.capture](#)]),
- (4.3) — naming the entity within the *compound-statement* of the innermost enclosing *lambda-expression* of P , but not in an unevaluated operand, would refer to an entity captured by copy in some intervening *lambda-expression*, and
- (4.4) — P is in the function parameter scope, but not the *parameter-declaration-clause*, of the innermost such *lambda-expression* E ,
- then the type of the expression is the type of a ~~class member access expression~~ (7.6.1.5 [[expr.ref](#)]) class-member-access-expression naming the non-static data member that would be declared for such a capture in the object parameter (9.3.4.6 [[decl.fct](#)]) of the function call operator of E .
- [Note: [. . .] — end note]
- [Example: [. . .] — end example]

§ 4.3.11 Destruction

[[expr.prim.id.dtor](#)]

Change 7.5.5.5 [[expr.prim.id.dtor](#)] paragraph 2 as follows:

- 2 If the *id-expression* names a pseudo-destructor, T shall be a scalar type and the *id-expression* shall appear as the right operand of a ~~class member access~~ (7.6.1.5 [[expr.ref](#)]) class-member-access-expression that forms the *postfix-expression* of a ~~function call~~ (7.6.1.3 [[expr.call](#)]) function-call-expression.

§ 4.3.12 Expression splicing

[[expr.prim.splice](#)]

Change 7.5.9 [[expr.prim.splice](#)] paragraph 4 as follows:

- 4 For a *splice-expression* of the form *template splice-specialization-specifier*, the *splice-specifier* of the *splice-specialization-specifier* shall designate a template *T*.

— [. . .]

[*Note*: Class members are accessible from any point when designated by *splice-expressions* (11.8.3 [class.access.base]). A ~~class-member-access-expression~~ (7.6.1.5) *class-member-access-expression* whose right operand is a *splice-expression* is ill-formed if the left operand (considered as a pointer) cannot be implicitly converted to a pointer to the designating class of the right operand. — *end note*]

§ 4.3.13 Unary operators

[expr.unary.op]

Change 7.6.2.2 [expr.unary.op] paragraph 3 as follows:

- 3 The operand of the unary *&* operator shall be an lvalue of some type *T*.

- (3.1) — If the operand is a *qualified-id* or *splice-expression* designating a non-static member *m*, other than an explicit object member function, *m* shall be a direct member of some class *C* that is not an anonymous union. The result has type “pointer to member of class *C* of type *T*” and designates *C::m*.

[*Note*: A *qualified-id* that names a member of a namespace-scope anonymous union is considered to be a ~~class-member-access-expression~~ (7.5.5.1) *class-member-access-expression* and cannot be used to form a pointer to member. — *end note*]

— [. . .]

§ 4.3.14 Comma operator

[expr.comma]

Change 7.6.20 [expr.comma] paragraph 2 as follows:

- 2 [*Note*: In contexts where the comma token is given special meaning (e.g., ~~function-calls~~ (7.6.1.3) *function-call-expressions*, ~~subscript-expressions~~ (7.6.1.2) *subscript-expressions*, ~~lists-of-initializers~~ (9.5) *initializer-lists*, or *template-argument-lists* (13.3 [temp.names])), the comma operator as described in this subclause can appear only in parentheses.

[*Example*: [. . .] — *end example*]

— *end note*]

§ 4.3.15 Core constant expressions

[expr.const.core]

Change [expr.const.core] paragraph 2 as follows:

- 2 An expression *E* is a *core constant expression* unless the evaluation of *E*, following the rules of the abstract machine (6.10.1 [intro.execution]), would evaluate one of the following:

- (2.1) — **this** (7.5.3 [expr.prim.this]), except
 - [. . .]
- (2.1.2) — when appearing as the *postfix-expression* of an implicit or explicit ~~class member access expression~~ (7.6.1.5 [expr.ref]) class-member-access-expression;
 - [. . .]
- (2.18) — an invocation of a destructor (11.4.7 [class.dtor]) or a **function call** function-call-expression whose *postfix-expression* names a pseudo-destructor (7.6.1.3 [expr.call]), in either case for an object whose lifetime did not begin within the evaluation of *E*;
 - [. . .]

§ 4.3.16 Default arguments

[dcl.fct.default]

Change 9.3.4.7 [dcl.fct.default] paragraph 9 as follows:

- 9 A default argument is evaluated each time the function is called with no argument for the corresponding parameter. A parameter shall not appear as a potentially evaluated expression in a default argument.

[Note: [. . .] — end note]

[Example: [. . .] — end example]

A non-static member shall not be designated in a default argument unless
 - (9.1) — it is designated by the *id-expression* or *splice-expression* of a ~~class member access expression~~ (7.6.1.5 [expr.ref]) class-member-access-expression,
 - [. . .]
- [. . .]

§ 4.3.17 Defaulted comparison operator functions

[class.compare.default]

Change 11.10.1 [class.compare.default] paragraph 5 as follows:

- 5 The direct base class subobjects of *C*, in the order of their declaration in the *base-specifier-list* of *C*, followed by the non-static data members of *C*, in the order of their declaration in the *member-specification* of *C*, form a list of subobjects. In that list, any subobject of array type is recursively expanded to the sequence of its elements, in the order of increasing subscript. Let $\mathbf{x}_i$ be an lvalue denoting the i^{th} element in the expanded list of subobjects for an object *x* (of length *n*), where $\mathbf{x}_i$ is formed by a sequence of derived-to-base conversions (12.2.4.2 [over.best.ics]), ~~class member access expressions~~ (7.6.1.5

~~[[expr.ref](#)])~~ [class-member-access-expressions](#), and ~~array subscript ex-~~
~~pressions~~ (7.6.1.2 [[expr.sub](#)]) [subscript-expressions](#) applied to `x`.

§ 4.3.18 Names of template specialization

[[temp.names](#)]

Change [[temp.names](#)] paragraph 3 as follows:

- 3 A `<` is interpreted as the delimiter of a *template-argument-list* if either
- (3.1) — it follows a *splice-specifier* that either
 - (3.1.1) — appears in a type-only context or
 - (3.1.1) — is preceded by `template` or `typename`, or
 - (3.2) — it follows a name that is not a *conversion-function-id* and
 - (3.2.1) — that follows the keyword `template` or a `~` after a *nested-name-specifier* or in a ~~class member access expression~~ [class-member-access-expression](#), or
 - [`. . .`]
- [Note: [`. . .`] — end note]
[Example: [`. . .`] — end example]

§ 4.3.19 Dependent names

[[temp.dep.general](#)]

Change 13.8.3.1 [[temp.dep.general](#)] paragraph 2 as follows:

- 2 A ~~dependent call~~ [dependent call](#) is ~~an expression~~ [a function-call-expression](#), possibly formed as a non-member candidate for an operator (12.2.2.3 [[over.match.oper](#)]), ~~of the form:~~
- ~~*postfix-expression* (~~*expression-list*~~_{*opt*})~~
- ~~where the~~ [whose](#) *postfix-expression* is an *unqualified-id* and
- (2.1) — any of the expressions in the *expression-list* is a pack expansion (13.7.4 [[temp.variadic](#)]), or
 - (2.2) — any of the expression or *brace-init-lists* in the *expression-list* is type-dependent (13.8.3.3 [[temp.dep.expr](#)]), or
 - (2.3) — the *unqualified-id* is a *template-id* in which any of the template arguments depends on a template parameters.
- The component name of an *unqualified-id* (7.5.5.2 [[expr.prim.id.unqual](#)]) is dependent if
- (2.4) — it is a *conversion-function-id* whose *conversion-type-id* is dependent, or
 - (2.5) — it is `operator=` and the current class is a templated entity, or
 - (2.6) — the *unqualified-id* is the *postfix-expression* in a dependent call.
- [Note: [`. . .`] — end note]

Do not change 13.8.3.3 [temp.dep.expr] paragraph 3:

- 3 [. . .] Expressions of the following forms are type-dependent only if the type specified by the *type-id*, *simple-type-specifier*, *typename-specifier*, or *new-type-id* is dependent, even if any subexpression is type-dependent:

```

simple-type-specifier ( expression-listopt )
simple-type-specifier braced-init-list
typename-specifier ( expression-listopt )
typename-specifier braced-init-list
::opt new new-placementopt new-type-id new-initializeropt
::opt new new-placementopt ( type-id ) new-initializeropt
dynamic_cast < type-id > ( expression )
static_cast < type-id > ( expression )
const_cast < type-id > ( expression )
reinterpret_cast < type-id > ( expression )
( type-id ) cast-expression

```

Change 13.8.3.3 [temp.dep.expr] paragraph 4 as follows:

- 4 Expressions of the following forms are never type-dependent (because the type of the expression cannot be dependent):

```

literal
sizeof unary-expression
sizeof ( type-id )
sizeof ... ( identifier )
alignof ( type-id )
typeid ( expression )
typeid ( type-id )
typeid-expression
::opt delete cast-expression
::opt delete [ ] cast-expression
throw assignment-expressionopt
noexcept ( expression )
requires-expression
reflect-expression

```

[Note: For the standard library macro `offsetof`, see 17.2 [support.types]. — end note]

Change 13.8.3.3 [temp.dep.expr] paragraph 5 as follows:

- 5 A ~~class member access expression~~ (7.6.1.5 [expr.ref]) class-member-access-expression is type-dependent if
- (5.1) — the terminal name of its *id-expression*, if any, is dependent,
 - (5.2) — its *splice-expression*, if any, is type-dependent, or

- (5.3) — the expression refers to a member of the current instantiation and the type of the referenced member is dependent.

§ 4.3.21 Value-dependent expression [temp.dep.constexpr]

Do not change 13.8.3.4 [temp.dep.constexpr] paragraphs 2 and 3:

- 2 [. . .] Expressions of the following form are value-dependent if the *unary-expression* or *expression* is type-dependent or the *type-id* is dependent:

```
sizeof unary-expression
sizeof ( type-id )
typeid ( expression )
typeid ( type-id )
alignof ( type-id )
```

[Note: For the standard library macro `offsetof`, see 17.2 [support.types]. — end note]

- 3 Expressions of the following form are value-dependent if either the *type-id*, *simple-type-specifier*, or *typename-specifier* is dependent or the *expression* or *cast-expression* is value-dependent or any *expression* in the *expression-list* is value-dependent or any *assignment-expression* in the *braced-init-list* is value-dependent:

```
simple-type-specifier ( expression-listopt )
typename-specifier ( expression-listopt )
simple-type-specifier braced-init-list
typename-specifier braced-init-list
static_cast < type-id > ( expression )
const_cast < type-id > ( expression )
reinterpret_cast < type-id > ( expression )
dynamic_cast < type-id > ( expression )
( type-id ) cast-expression
```

§ 4.3.22 Deducing template arguments from a type [temp.deduct.type]

Change 13.10.3.6 [temp.deduct.type] paragraph 23 as follows:

- 23 The *template-argument* corresponding to a template template parameter is deduced from the type of the *template-argument* of a class template specialization used in the argument list of a function call (12.2.2.2.2 [over.call.func]).
[Example: [. . .] — end example]

§ 4.3.23 Exception specifications [except.spec]

Change 14.5 [except.spec] paragraph 5 as follows:

- 5 An expression *E* is *potentially-throwing* if

- (5.1) — *E* is a ~~function call (7.6.1.3 [expr.call])~~ *function-call-expression* whose *postfix-expression* has a function type, or a pointer-to-function type, with a potentially-throwing exception specification, or
- [. . .]

§ 4.4 Library wording

§ 4.4.1 Algorithm function objects

[alg.func.obj]

Change 16.3.3.4 [alg.func.obj] paragraph 2 as follows:

- 2 For an algorithm function object *o*, let *S* be the corresponding set of function templates. Then for any sequence of arguments *args* . . ., *o*(*args* . . .) is expression-equivalent to *s*(*args* . . .), where the result of name lookup for *s* is the overload set *S*.

[*Note*: Algorithm function objects are not found by argument-dependent name lookup (6.5.4 [basic.lookup.argdep]). When found by unqualified name lookup (6.5.3 [basic.lookup.unqual]) for the *postfix-expression* in a ~~function call (7.6.1.3 [expr.call])~~ *function-call-expression*, they inhibit argument-dependent name lookup.

[*Example*:

```
void foo() {
    using namespace std::ranges;
    std::vector<int> vec{1,2,3};
    find(begin(vec), end(vec), 2);           // #1
}
```

The ~~function-call-expression~~ *function-call-expression* at #1 invokes `std::ranges::find`, not `std::find`. — *end example*]

— *end note*]

§ 4.4.2 Creation

[support.srcloc.cons]

Change 17.8.2.2 [support.srcloc.cons] paragraph 1 as follows:

```
static consteval source_location current() noexcept;
```

- 1 *Returns:*

- (1.1) — When invoked by a ~~function call~~ *function-call-expression* whose *postfix-expression* is a (possibly parenthesized) *id-expression* naming `current`, returns a `source_location` with an implementation-defined value. [. . .]
- [. . .]

Change 22.10.1 [function.objects.general] paragraph 1 as follows:

- ¹ A function object type is an object type (6.9.1 [basic.types.general]) that can be the type of the *postfix-expression* in a ~~function call~~ ~~(7.6.1.3 [expr.call], 12.2.2.2 [over.match.call])~~ function-call-expression.¹⁷⁴

§ 4.5 Annex C

§ 4.5.1 Clause 7: expressions

[diff.cpp20.expr]

Change C.2.3 [diff.cpp20.expr] paragraph 2 as follows:

Affected subclause: 7.6.1.2 [expr.sub]

Change: Change the meaning of comma in ~~subscript expressions~~ subscript-expressions.

Rationale: Enable repurposing a deprecated syntax to support multidimensional indexing.

Effect on original feature: Valid C++ 2020 code that uses a comma expression within a ~~subscript expression~~ subscript-expression may fail to compile.

[Example:

```
arr[1, 2] // was equivalent to arr[(1, 2)],
          // now equivalent to arr.operator[](1, 2) or ill-formed
```

— end example]